Laboratório de Criptografia

Criptografia e Segurança Informática | Guião prático

Unidade Curricular: Criptografia e Segurança Informática

Modalidade: Trabalho prático em grupos de 2 estudantes

Linguagem: Python 3

Nível: Introdutório / Intermédio

Sobre o projeto

Bem-vindos ao **Laboratório de Criptografia**!

Neste laboratório vão assumir o papel de uma equipa de analistas de cibersegurança contratada para avaliar diferentes mecanismos de proteção da informação.

A missão consiste em implementar algoritmos de cifragem, quebrar mensagens secretas, identificar vulnerabilidades e analisar o comportamento de diferentes modos de operação criptográficos.

Ao longo do laboratório vão explorar:

• Criptografia clássica e cifra de César

• Ataques por força bruta

• Criptografia simétrica com AES

• Modos de operação ECB, CBC, CFB e OFB

• Segurança, confidencialidade e integridade

O objetivo não é apenas fazer o código funcionar, mas compreender o que acontece aos dados e porquê.

Preparação do ambiente

Requisitos

• Python 3

• Visual Studio Code, PyCharm ou outro editor

• Biblioteca `cryptography`

Os códigos iniciais são pontos de partida: as tarefas assinaladas com TODO devem ser concluídas pelo grupo. Executem os comandos a partir da pasta do projeto. Em sistemas em que o comando é `python3`, utilizem-no em vez de `python`.

Instalação

Verificar a versão do Python:

```
python --version
```

Instalar a biblioteca necessária:

```
python -m pip install cryptography
```

Estrutura sugerida do projeto

```
cryptography-lab/
+-- README.md
+-- challenge-01/
|   +-- caesar_breaker.py
+-- challenge-02/
|   +-- cipher_tool.py
+-- challenge-03/
|   +-- block_ciphers.py
+-- challenge-04/
|   +-- aes_modes.py
+-- challenge-05/
|   +-- modes_comparison.py
+-- report/
    +-- report.pdf
```

Desafio 01 - César: ataque por força bruta

Objetivo: Recuperar uma mensagem secreta utilizando técnicas de criptoanálise.

Cenário

Durante uma investigação de segurança, a equipa intercetou a seguinte mensagem:

```
D VHJXUDQFD GRV GDGRV H HVVHQFLDO
```

Sabe-se que foi utilizada a **cifra de César**, mas a chave é desconhecida. Neste desafio, o alfabeto é A-Z, sem acentos. Testem as 26 chaves, de 0 a 25, incluindo a transformação identidade.

A vossa missão

1. Desenvolver um programa capaz de decifrar mensagens com a cifra de César.
2. Implementar um ataque por força bruta que teste todas as chaves possíveis.
3. Identificar a chave utilizada.
4. Recuperar a mensagem original.
5. Explicar por que razão este algoritmo não é considerado seguro atualmente.

Código inicial

```
mensagem = "D VHJXUDQFD GRV GDGRV H HVVHQFLDO"

def decifrar_cesar(texto, chave):
    resultado = ""

    for letra in texto:
        if 'A' <= letra <= 'Z':
            pos = ord(letra) - ord('A')
            nova_pos = (pos - chave) % 26
            resultado += chr(nova_pos + ord('A'))
        else:
            resultado += letra

    return resultado

# TODO: Implementar o ataque por força bruta
# TODO: Apresentar todas as chaves possíveis
# TODO: Identificar a mensagem original
```

Entregável: `challenge-01/caesar_breaker.py`

Desafio 02 - Construir uma ferramenta César

Objetivo: Desenvolver uma ferramenta de cifragem e decifragem.

Cenário

A empresa pretende disponibilizar uma ferramenta simples para demonstrar o funcionamento da criptografia clássica.

A vossa equipa deverá desenvolver uma aplicação de consola em Python.

Funcionalidades obrigatórias

A aplicação deve apresentar o seguinte menu:

```
================================
       FERRAMENTA DE CRIPTOGRAFIA
================================

[1] Cifrar mensagem
[2] Decifrar mensagem
[3] Ataque por força bruta
[4] Sair

Escolha uma opção:
```

Requisitos

• Permitir introduzir mensagens personalizadas.

• Permitir escolher a chave de cifragem.

• Implementar a cifragem e a decifragem.

• Preservar espaços, pontuação e maiúsculas/minúsculas.

• Deslocar apenas A-Z e a-z; manter os restantes caracteres, incluindo letras acentuadas, inalterados.

• Validar opções do menu e chaves inteiras, normalizando a chave com módulo 26.

• Implementar um ataque automático por força bruta.

• Permitir executar várias operações sem reiniciar a aplicação.

Testes obrigatórios

1. Cifrar três mensagens diferentes.
2. Utilizar chaves distintas.
3. Confirmar que a decifragem recupera as mensagens originais.
4. Trocar uma mensagem cifrada com outro grupo.
5. Tentar recuperar a mensagem recebida sem conhecer a chave.

Questão de investigação

Qual é a principal vulnerabilidade da cifra de César e de que forma poderia ser explorada por um atacante?

Código inicial

```
def transformar(texto, chave):
    resultado = ""
    for letra in texto:
        if "A" <= letra <= "Z":
            base = ord("A")
        elif "a" <= letra <= "z":
            base = ord("a")
        else:
            resultado += letra
            continue
        resultado += chr((ord(letra) - base + chave) % 26 + base)
    return resultado

# Cifrar: transformar(texto, chave)
# Decifrar: transformar(texto, -chave)
# TODO: Criar o menu, validar entradas e repetir ate sair.
# TODO: Integrar o ataque por forca bruta do desafio 01.
```

Entregável: `challenge-02/cipher_tool.py`

Desafio 03 - DES, AES, blocos e padding

Objetivo: Compreender o funcionamento das cifras por blocos e investigar as diferenças entre os algoritmos DES e AES.

Cenário

Uma empresa utiliza diferentes algoritmos criptográficos para proteger informação confidencial.

Durante uma auditoria, foram identificados sistemas que ainda utilizam DES e outros que já utilizam AES.

A vossa missão é investigar as características destes algoritmos e compreender por que razão alguns mecanismos deixaram de ser considerados seguros.

Parte A - Investigação

Pesquisar e identificar as seguintes características:

Característica	DES	AES
Tipo de criptografia		
Tamanho do bloco		
Tamanho da chave		
Número de rondas		
Segurança atual		

Parte B - Análise de blocos

Desenvolver um programa Python que:

1. Solicite ao utilizador uma mensagem.
2. Converta a mensagem para bytes.
3. Divida a mensagem em blocos de 16 bytes.
4. Apresente os blocos individualmente.
5. Identifique se o último bloco está completo.

6. Aplique PKCS#7 para blocos de 16 bytes e apresente os blocos resultantes.

No PKCS#7, acrescentam-se n bytes com valor n. Se os dados já forem múltiplos de 16 bytes (ou estiverem vazios), acrescenta-se um bloco completo de 16 bytes. O tamanho deve ser medido em bytes, não em caracteres. Ao decifrar, o preenchimento deve ser validado e removido.

Código inicial

```
mensagem = input("Introduza uma mensagem: ")

dados = mensagem.encode("utf-8")

tamanho_bloco = 16

for i in range(0, len(dados), tamanho_bloco):
    bloco = dados[i:i + tamanho_bloco]
    print(f"Bloco: {bloco}")

# TODO: Identificar blocos incompletos
# TODO: Aplicar PKCS#7: n = 16 - len(dados) % 16
# TODO: Acrescentar bytes([n]) * n
# TODO: Apresentar o resultado final
```

Parte C - Questões de investigação

1. Qual é a principal diferença entre DES e AES?
2. Porque é que o DES deixou de ser considerado seguro?
3. Porque é necessário utilizar padding em determinados modos de cifragem por blocos?
4. Qual é a diferença entre o tamanho do bloco e o tamanho da chave?
5. O AES é considerado uma cifra simétrica ou assimétrica? Justificar.

Entregável: `challenge-03/block_ciphers.py`

Desafio 04 - AES: ECB e CBC

Objetivo: Investigar o comportamento do AES em diferentes modos de operação e identificar possíveis fragilidades.

Cenário

A empresa utiliza AES para proteger informação confidencial.

A equipa de segurança identificou que alguns sistemas utilizam ECB, enquanto outros utilizam CBC.

Será necessário investigar as diferenças entre os dois modos.

Parte A - Implementação

```
from cryptography.hazmat.primitives.ciphers import (
    Cipher, algorithms, modes
)
import os

chave = os.urandom(16)
iv = os.urandom(16)

dados = b"A" * 64

def cifrar(modo):
    cipher = Cipher(algorithms.AES(chave), modo)
    encryptor = cipher.encryptor()
    return encryptor.update(dados) + encryptor.finalize()

ecb = cifrar(modes.ECB())
cbc = cifrar(modes.CBC(iv))

print("ECB:", ecb.hex())
print("CBC:", cbc.hex())

# TODO: Dividir os resultados em blocos
# TODO: Identificar blocos repetidos
# TODO: Comparar ECB e CBC
```

Parte B - Experiências

Os dados iniciais já têm comprimento múltiplo de 16 bytes e não precisam de padding nesta experiência. Para mensagens arbitrárias em ECB/CBC, utilizem PKCS#7 e removam-no após a decifragem. Criem um novo contexto para cada operação.

Experiência 1 - Repetição de blocos

Dividir os resultados em blocos de 16 bytes e identificar blocos repetidos.

Experiência 2 - Alteração do IV

Cifrar novamente os mesmos dados com CBC, utilizando outro IV.

Experiência 3 - Alteração da mensagem

Alterar um único byte da mensagem e comparar os resultados obtidos, mantendo a mesma chave e o mesmo IV apenas nesta comparação controlada. Nas restantes experiências, gerar um novo IV para cada cifragem CBC.

Experiência 4 - Comparação

Analisar as diferenças entre os criptogramas produzidos por ECB e CBC.

Questões de investigação

1. Porque é que o ECB pode revelar padrões existentes no texto original?
2. Qual é a função do IV no modo CBC?
3. O que acontece quando a mesma mensagem é cifrada com a mesma chave e o mesmo IV?
4. Qual dos modos oferece melhor proteção contra a exposição de padrões?
5. A utilização de CBC garante que os dados não foram alterados? Justificar.

Desafio extra - Imagens cifradas

Selecionar uma imagem BMP de teste e investigar o que acontece quando os seus dados são cifrados utilizando ECB e CBC.

Para permitir a visualização, podem preservar o cabeçalho BMP e cifrar apenas os dados da imagem, garantindo que o formato e o alinhamento dos blocos são respeitados.

Comparar visualmente as imagens produzidas e explicar os resultados.

Nota: ECB e CBC são utilizados aqui para fins de aprendizagem. Para proteger informação real, devem ser utilizados mecanismos de cifragem autenticada adequados, como AES-GCM.

Entregável: `challenge-04/aes_modes.py`

Desafio 05 - CFB, OFB e análise de segurança

Objetivo: Comparar diferentes modos de operação e avaliar o seu comportamento perante alterações aos criptogramas.

Cenário

A empresa pretende selecionar mecanismos criptográficos para proteger diferentes tipos de informação.

A equipa deverá investigar os modos:

• ECB - Electronic Codebook

• CBC - Cipher Block Chaining

• CFB - Cipher Feedback

• OFB - Output Feedback

Parte A - Implementação

Adaptar o código desenvolvido anteriormente para experimentar CFB e OFB.

Para cada modo:

1. Cifrar uma mensagem.
2. Decifrar o criptograma.
3. Confirmar que a mensagem original é recuperada.
4. Alterar um bit do criptograma.
5. Observar o resultado da decifragem após essa alteração.
6. Registar e comparar os resultados.

Código inicial para CFB e OFB

```
import os
from cryptography.hazmat.primitives.ciphers import (
    Cipher, algorithms
)
try:
    from cryptography.hazmat.decrepit.ciphers.modes import CFB, OFB
except ImportError:
    from cryptography.hazmat.primitives.ciphers.modes import CFB, OFB

chave = os.urandom(16)
dados = b"Mensagem de teste. " * 4

for Modo in (CFB, OFB):
    iv = os.urandom(16)
    enc = Cipher(algorithms.AES(chave), Modo(iv)).encryptor()
    ct = enc.update(dados) + enc.finalize()
    dec = Cipher(algorithms.AES(chave), Modo(iv)).decryptor()
    recuperado = dec.update(ct) + dec.finalize()
    assert recuperado == dados
    print(Modo.__name__, ct.hex())
    # TODO: Alterar um bit e decifrar com um novo contexto.
    # TODO: Comparar os bytes recuperados com os originais.
```

O código utiliza CFB com feedback de bloco completo (128 bits), não CFB8. CFB e OFB não necessitam de padding. A localização destas classes depende da versão instalada da biblioteca; a importação alternativa

permite usar ambas as localizações.

Para alterar exatamente um bit sem operações bit a bit, convertam o criptograma em `bytearray`, escolham uma posição i válida e apliquem `alterado[i] += 1 if alterado[i] % 2 == 0 else -1`. Isto altera apenas o bit menos significativo do byte escolhido. Decifrem `bytes(alterado)` com a mesma chave e IV da mensagem original e um novo contexto.

No teste de propagação, usem dados com pelo menos quatro blocos e alterem um byte do primeiro bloco. Em ECB/CBC, comparem primeiro os bytes decifrados antes de remover o padding; registem separadamente eventuais erros na sua validação. Não convertam bytes corrompidos diretamente para UTF-8: apresentem-nos em hexadecimal.

Parte B - Comparação

Completar a tabela:

Característica	ECB	CBC	CFB	OFB
Necessita de IV?				
Oculta padrões repetidos?				
Necessita de padding?				
Utiliza encadeamento ou feedback?				
Como se propaga a alteração de um bit?				

Parte C - Decisão de segurança

Uma empresa pretende proteger:

Cenário A - Ficheiros confidenciais

Documentos confidenciais armazenados num servidor.

Cenário B - Comunicação em rede

Mensagens transmitidas através de uma rede.

Cenário C - Integridade dos dados

Informação cuja alteração não autorizada deve ser detetada.

Para cada cenário, devem:

1. Identificar os requisitos de segurança.
2. Selecionar um mecanismo criptográfico adequado.
3. Justificar a escolha.
4. Identificar possíveis limitações ou vulnerabilidades.

A análise deve distinguir claramente os conceitos de confidencialidade, integridade e autenticação. Nenhum dos quatro modos estudados autentica os dados por si só. Justifiquem quando é necessário recorrer a cifragem autenticada. Os IV não são secretos: devem acompanhar o criptograma. Evitem a sua reutilização com a mesma chave; CBC e CFB requerem IV imprevisível e OFB requer IV único.

Entregável: `challenge-05/modes_comparison.py`

Submissão

Cada grupo deverá organizar o projeto de acordo com a estrutura sugerida e submeter um ficheiro `.zip` no Moodle.

O projeto deverá conter:

Código-fonte

Todos os ficheiros Python desenvolvidos, organizados por desafio.

Evidências

Capturas de ecrã que demonstrem a execução das aplicações e os resultados das experiências.

Relatório técnico

Um relatório PDF contendo:

• Identificação dos elementos do grupo.

• Respostas às questões de investigação.

• Resultados e observações das experiências.

• Tabela comparativa dos modos de operação.

• Conclusão

O código deve estar devidamente organizado, comentado quando necessário e ser executável.

Incluam no README os comandos para executar cada desafio e a versão de Python e de `cryptography` utilizadas. Guardem as capturas numa pasta `evidence/` e o relatório em `report/report.pdf`. Antes da submissão, extraiam o ZIP numa nova pasta e confirmem que os programas executam seguindo essas instruções. Não incluam o ambiente virtual nem chaves ou dados pessoais reais.

Missão final

Após a realização de todos os desafios, respondam à seguinte questão:

Se fossem responsáveis pela segurança de uma aplicação que processa informação confidencial, que mecanismos criptográficos escolheriam e porquê?

A resposta deverá considerar os conhecimentos adquiridos durante o laboratório, as vulnerabilidades identificadas e a importância de selecionar mecanismos adequados aos requisitos de segurança.

Laboratório de Criptografia | Criptografia e Segurança Informática

Pensar. Programar. Cifrar. Analisar. Compreender.

Referências de apoio

• Documentação oficial da biblioteca cryptography - cifragem simétrica: https://cryptography.io/en/latest/hazmat/primitives/symmetric-encryption/

• NIST, FIPS 197 - Advanced Encryption Standard (AES): https://csrc.nist.gov/pubs/fips/197/final

• NIST, SP 800-38A - modos de operação: https://csrc.nist.gov/pubs/sp/800/38/a/final

Consultem fontes técnicas oficiais e identifiquem no relatório as fontes utilizadas na investigação.